MINGGU 7 PENGOLAHAN CITRA DIGITAL (3 SKS / 150 MENIT)

Minggu ke-7: Teknik Thresholding dan Segmentasi Citra Digital

Target Pembelajaran (Sub-CPMK 7):
Mahasiswa mampu menerapkan teknik segmentasi citra menggunakan metode Global Thresholding (Otsu), Adaptive Thresholding (Mean & Gaussian), Segmentasi Warna HSV, serta Watershed Algorithm untuk pemisahan objek yang saling menempel secara komputasional dengan Python.

Endah Kristiani, Ph.D.

Struktur Perkuliahan & Alokasi Waktu (150 Menit)

Sesi 1: Teori Thresholding Global (Otsu) vs Adaptif & Watershed (50 Menit)

Fokus Aktivitas: Maksimasi varians antar-kelas Otsu, masalah gradasi bayangan pada Adaptive Thresholding, dan topografi Watershed pemisah sel.

Target Capaian: Mahasiswa memahami formulasi matematis pemisahan wilayah homogen.

Sesi 2: Studi Kasus Nyata & Bedah Kode Python (50 Menit)

Fokus Aktivitas: Studi kasus faktur berbayangan, penghitung sel darah mikroskopis, klasifikasi buah apel-jeruk, live coding cv2.threshold, cv2.adaptiveThreshold, cv2.watershed.

Target Capaian: Mahasiswa mampu memisahkan dan menghitung jumlah objek otomatis.

Sesi 3: Praktikum Mandiri Mahasiswa & Studi Kasus (50 Menit)

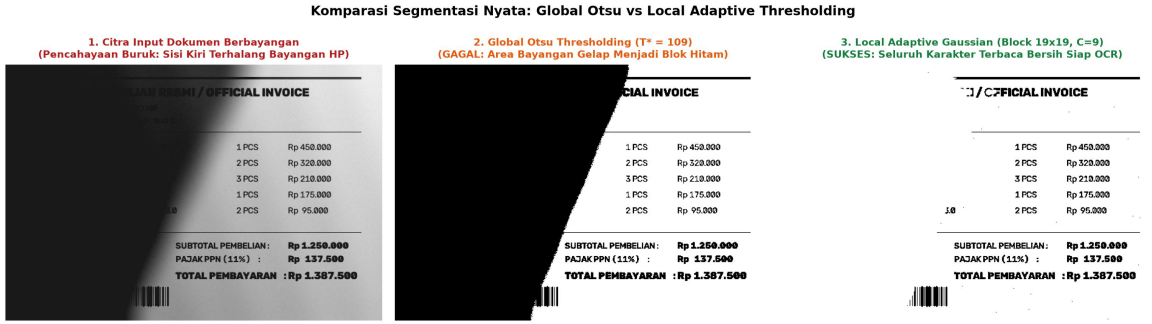
Fokus Aktivitas: Mahasiswa membangun sistem binarisasi dokumen berbayangan dan penghitung sel darah/koin otomatis.

Target Capaian: Source code Python mandiri dan visualisasi marker watershed.

Global Otsu vs Local Adaptive Thresholding

Otsu vs Adaptive Thresholding

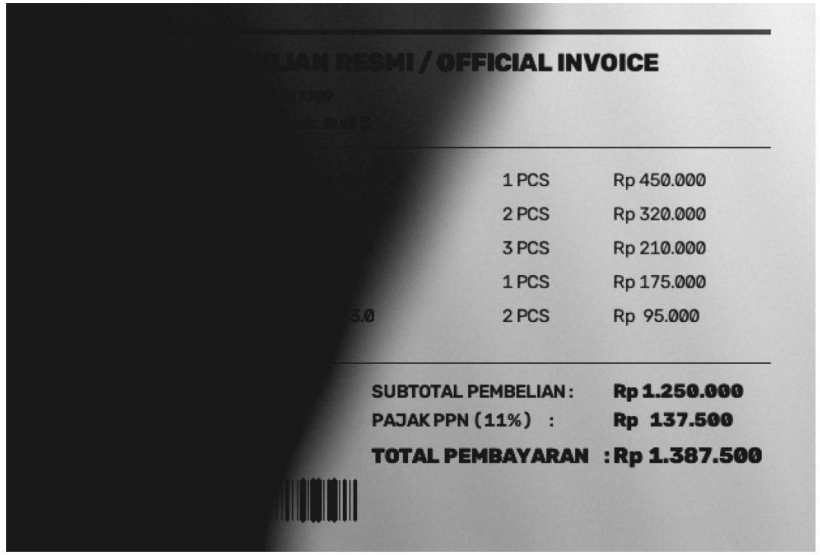
- 1. Otsu's Optimum Global Thresholding:
  - Mencari ambang tunggal T* yang memaksimalkan varians antar-kelas: sigma_B^2(T) = w0 . w1 . (mu0 - mu1)^2.
  - Sangat efektif pada citra dengan histogram bimodal (2 puncak jelas).
  - GAGAL TOTAL jika pencahayaan tidak merata (terdapat gradien bayangan).
- 2. Local Adaptive Thresholding (Block-Based):
  - Menghitung nilai ambang independen T(x, y) untuk setiap jendela lokal (misal 19x19 piksel) berdasarkan rata-rata terbobot Gaussian.
  - STANDAR EMAS untuk OCR dokumen kertas yang terkena bayangan smartphone / lampu!



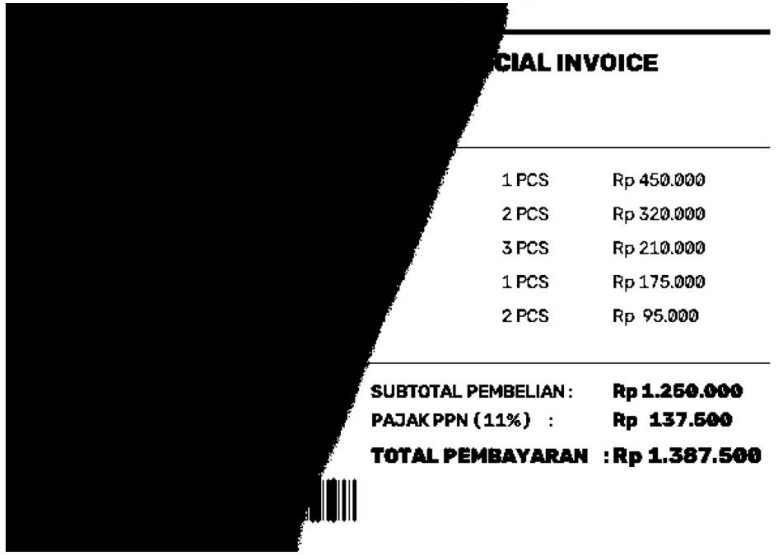
Global Otsu vs Local Adaptive Thresholding

Komparasi Segmentasi Nyata: Global Otsu vs Local Adaptive Thresholding

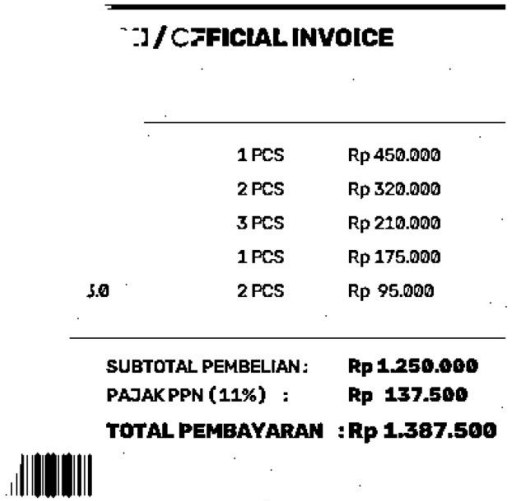
1. Citra Input Dokumen Berbayangan
(Pencahayaan Buruk: Sisi Kiri Terhalang Bayangan HP)



2. Global Otsu Thresholding (T* = 109)
(GAGAL: Area Bayangan Gelap Menjadi Blok Hitam)



3. Local Adaptive Gaussian (Block 19x19, C=9)
(SUKSES: Seluruh Karakter Terbaca Bersih Siap OCR)



Tabel Perbandingan Global Otsu dan Local Adaptive Thresholding

Kriteria Evaluasi	Global Otsu's Thresholding	Local Adaptive Thresholding
Cakupan Nilai Ambang	Satu nilai tunggal (T^*) untuk seluruh citra	Nilai unik $T(x, y)$ untuk setiap piksel
Asumsi Bentuk Histogram	Membutuhkan histogram bimodal (2 puncak terpisah jelas)	Bebas asumsi histogram; bekerja baik pada histogram unimodal/multimodal
Ketahanan Pencahayaan	Sangat buruk terhadap bayangan, gradien cahaya, dan vignetting	Sangat tangguh (robust) terhadap pencahayaan tidak merata & bayangan ekstrem
Kompleksitas Komputasi	Sangat ringan: $\mathcal{O}(L)$ dengan $L = 256$ tingkat keabuan	Sedikit lebih berat: $\mathcal{O}(N \times B)$ (namun sangat cepat dengan Integral Image)
Parameter Input	Otomatis 100% (tidak perlu tuning parameter)	Perlu menentukan Block Size (B) dan Konstanta (C)
Perilaku di Area Polos	Bersih sempurna di area latar belakang homogen	Jika C terlalu kecil, dapat memunculkan bintik derau di area kosong
Skenario Terbaik	• Dokumen hasil flatbed scanner • Siluet kontras tinggi • Citra industri dengan lampu studio merata	• Foto dokumen dari kamera smartphone • Pembacaan plat nomor (ANPR) malam hari • Naskah kuno / dokumen arsip yang bernoda/lusuh

Tips Menentukan Parameter Adaptive Thresholding:

1. **Block Size (B):** Harus bernilai **ganjil** dan berukuran **lebih besar daripada ketebalan huruf/garis objek**, tetapi **lebih kecil daripada skala gradasi bayangan** (rekomendasi: $15,19,25,31$).
2. **Konstanta Offset (C):**
    1. Jika latar belakang putih masih menyisakan bintik-bintik hitam derau $\rightarrow$ **Naikkan nilai** C (misal $C = 8 \ldots 12$).
    2. Jika goresan teks tampak terlalu tipis atau terputus-putus $\rightarrow$ **Turunkan nilai** C (misal $C = 2 \ldots 5$).

Watershed Algorithm: Pemisahan Objek yang Saling Menempel

Pipeline Marker Watershed

• Masalah: Pada penghitungan sel darah / butiran koin, objek sering saling bertumpuk (touching/overlapping objects) sehingga binarisasi biasa menganggapnya sebagai 1 gumpalan raksasa.

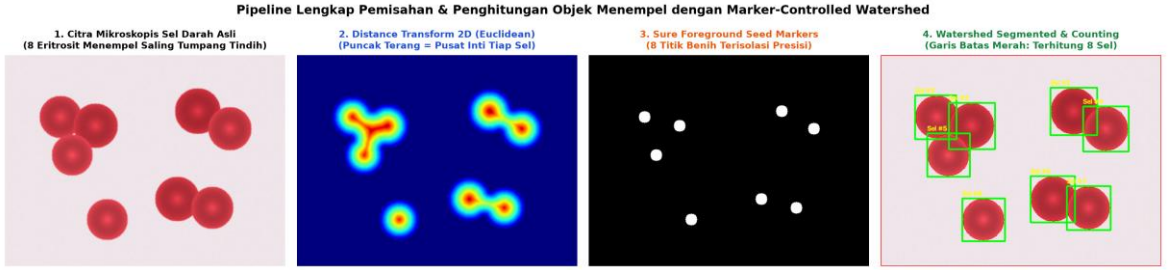
• Solusi Marker-Controlled Watershed:

1. Binarisasi Awal & Pembersihan Morfologi Opening.

2. Distance Transform 2D: Menghitung jarak Euclidean setiap piksel putih ke latar hitam terdekat (pusat tiap sel menjadi puncak terang).

3. Thresholding Distance Transform untuk menemukan titik penanda pasti (Sure Foreground Seed Markers).

4. Algoritma Watershed menggenangi air dari titik marker hingga bendungan (ridge lines) terbentuk tepat di batas pertemuan antar-sel.



Watershed Algorithm: Pemisahan Objek yang Saling Menempel

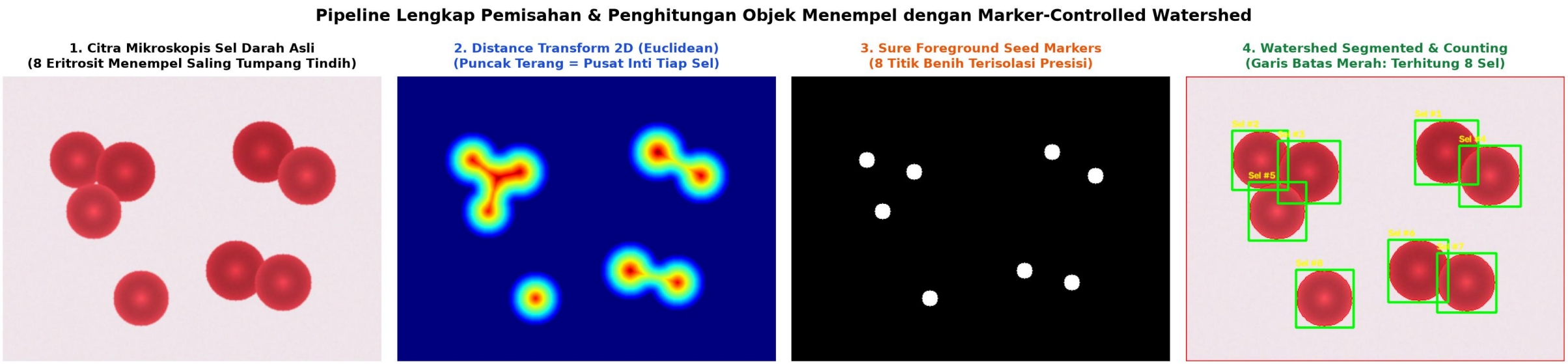
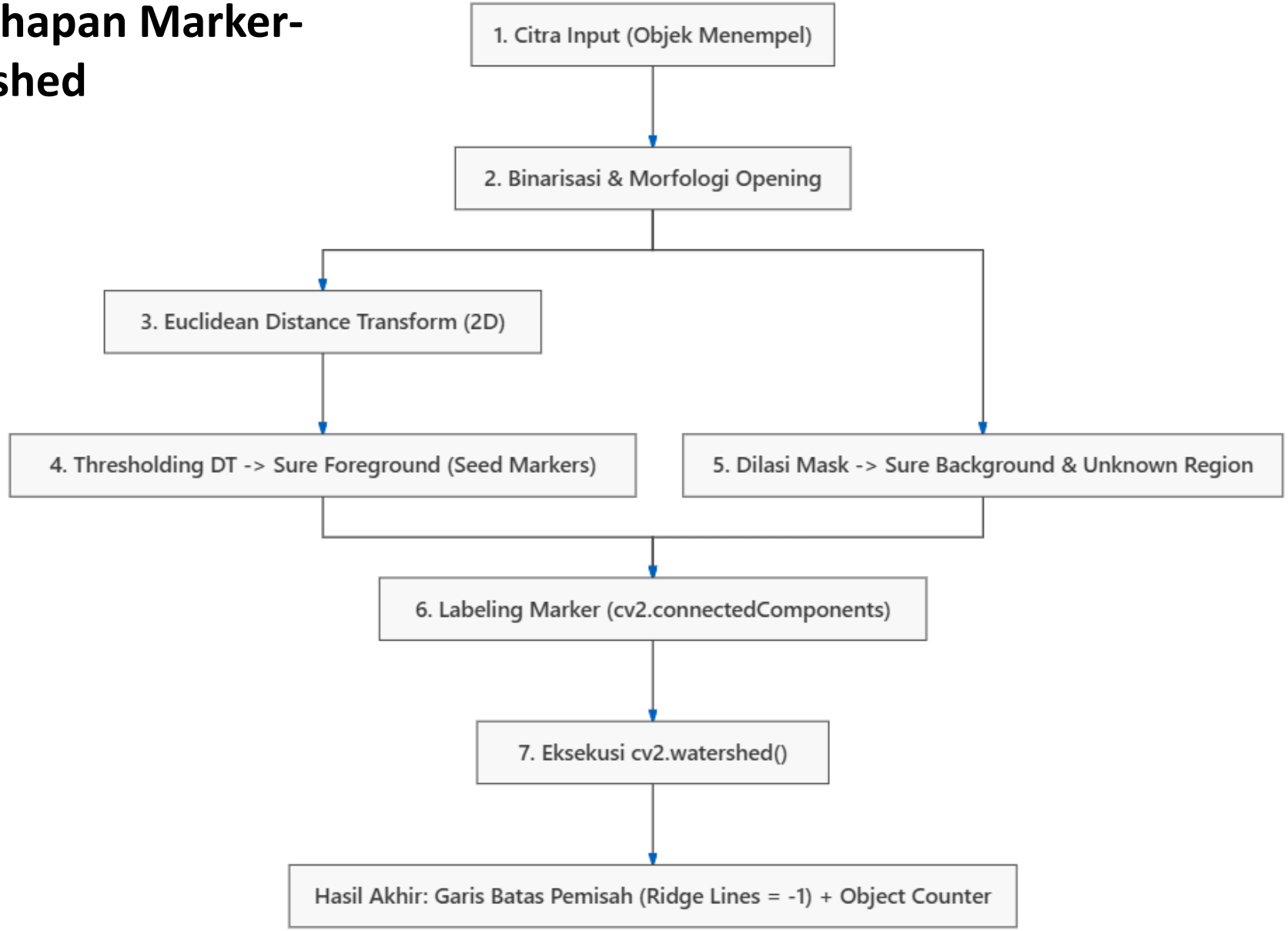
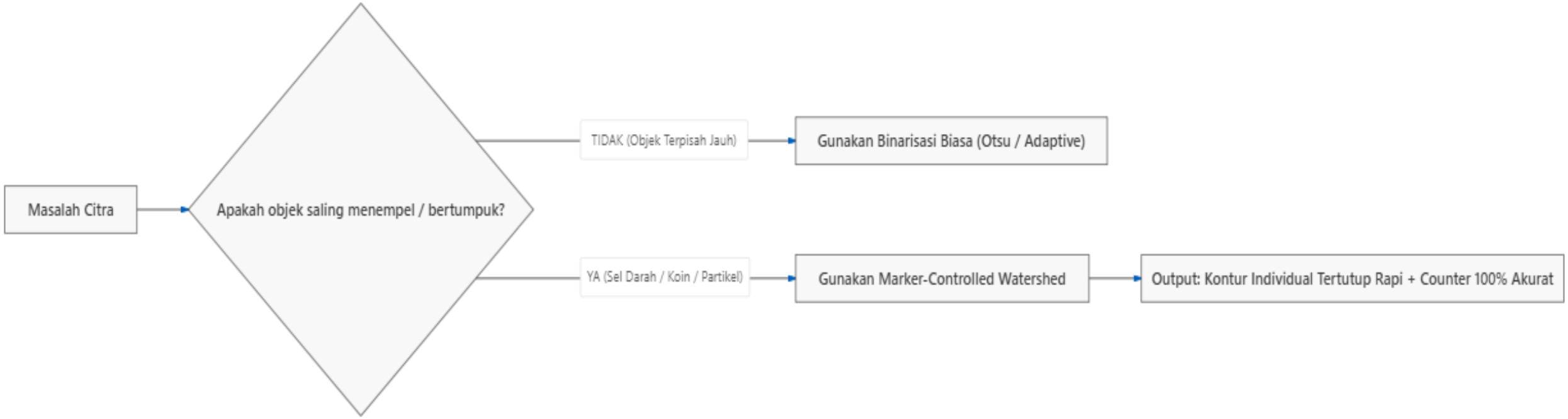


Diagram Alur 6 Tahapan Marker-Controlled Watershed



Skenario Penggunaan

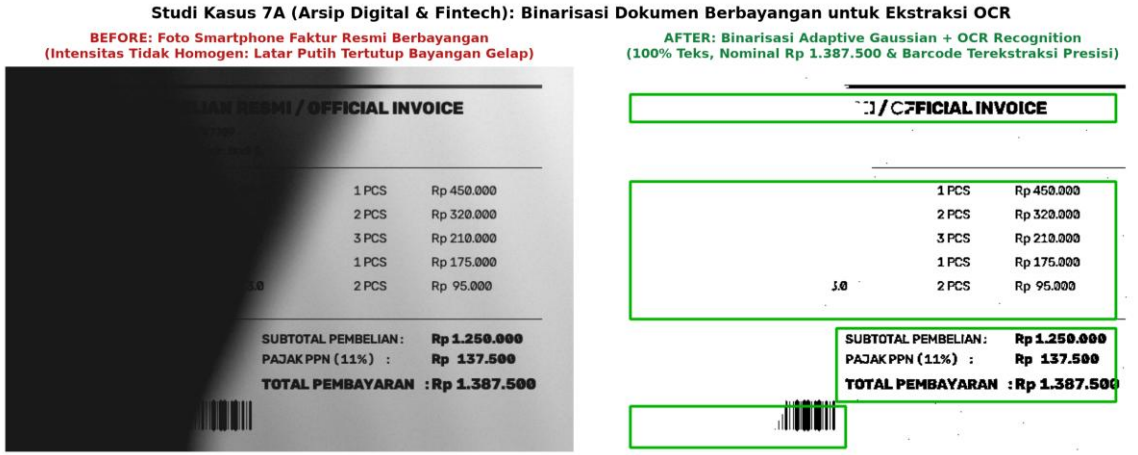


- **Laboratorium Hematologi & Patologi:** Menghitung jumlah eritrosit (*RBC*), leukosit (*WBC*), dan sel kanker secara otomatis tanpa terganggu tumpang tindih preparat.
- **Industri Farmasi & Manufaktur:** Menghitung jumlah butiran tablet/kapsul dan menginspeksi cacat geometri pada baut/mur yang bersentuhan di ban berjalan.
- **Metalurgi & Geologi:** Segmentasi butiran kristal mineral dan batuan mikroskopis (*grain size analysis*).

Studi Kasus 7A (Arsip Digital): Binarisasi Faktur Berbayangan (Adaptive)

Binarisasi Dokumen Berbayangan

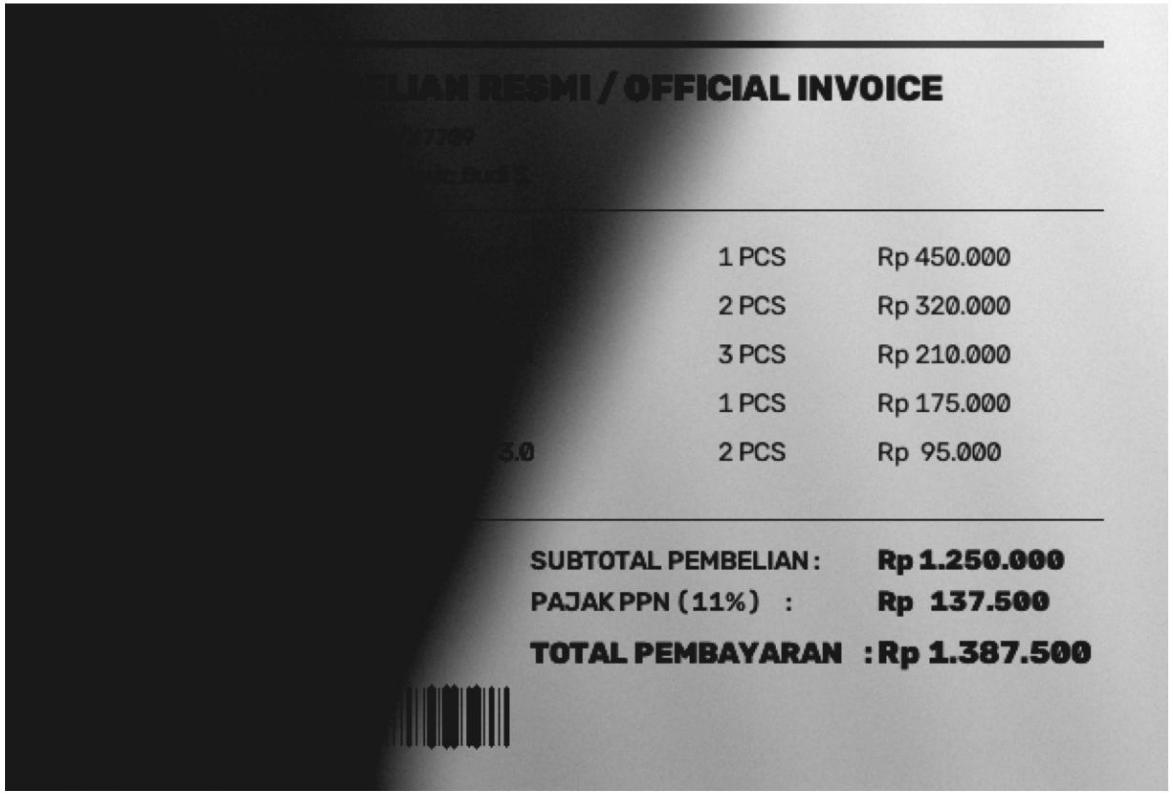
- Masalah (Before): Pengguna memfoto faktur resmi dengan bayangan smartphone menutupi separuh kertas. Global Otsu gagal total karena mengubah area bayangan menjadi blok hitam pekat.
- Solusi (Adaptive Gaussian Thresholding):
  - Menghitung nilai ambang lokal T(x, y) untuk tiap jendela 19x19 piksel.
  - Mengkompensasi gradasi bayangan secara dinamis di seluruh halaman dokumen.
- Hasil (After): Teks 'FAKTUR PEMBELIAN RESMI: RP 1.387.500' dan Barcode tersegmentasi biner bersih 100%, siap diekstraksi oleh sistem OCR perbankan.



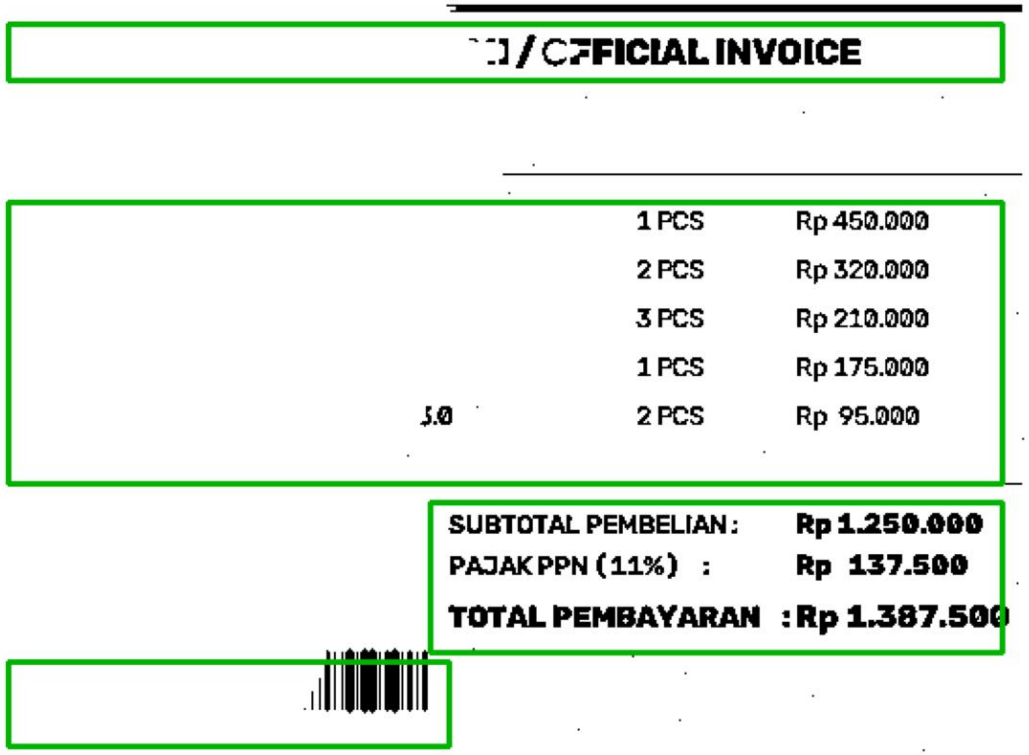
Studi Kasus 7A (Arsip Digital): Binarisasi Faktur Berbayangan (Adaptive)

Studi Kasus 7A (Arsip Digital & Fintech): Binarisasi Dokumen Berbayangan untuk Ekstraksi OCR

BEFORE: Foto Smartphone Faktur Resmi Berbayangan
(Intensitas Tidak Homogen: Latar Putih Tertutup Bayangan Gelap)



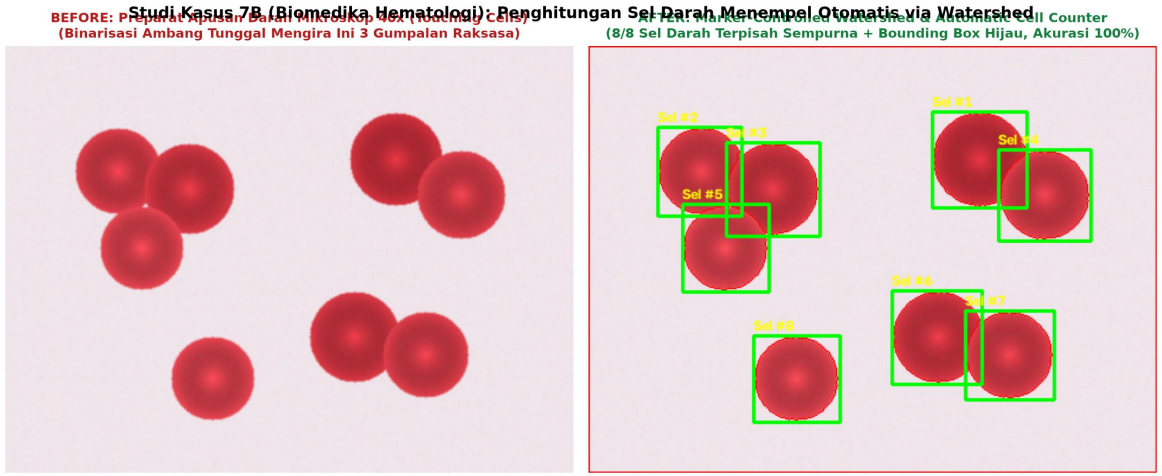
AFTER: Binarisasi Adaptive Gaussian + OCR Recognition
(100% Teks, Nominal Rp 1.387.500 & Barcode Terekstraksi Presisi)



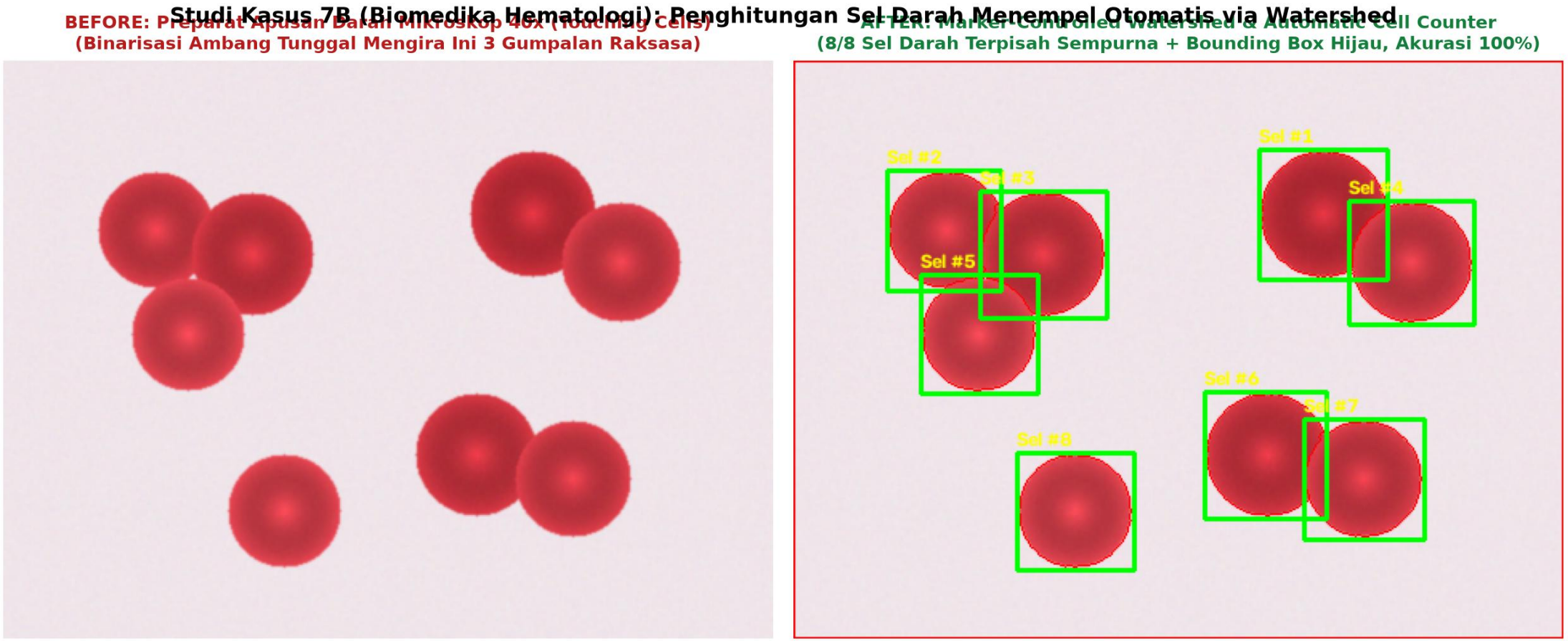
Studi Kasus 7B (Biomedika): Penghitungan Sel Darah Menempel (Watershed)

Pemisahan Sel Darah Tumpang Tindih

- Masalah (Before): Pada citra mikroskop laboratorium hematologi, sel-sel darah sering saling bertumpuk (touching erythrocytes). Binarisasi konvensional mengira 8 sel menempel adalah 3 gumpalan besar.
- Solusi (Marker-Controlled Watershed):
  - Distance Transform 2D menemukan pusat kedalaman tiap sel.
  - Watershed menarik garis batas merah pemisah tepat di titik perlekatan.
- Hasil (After): Sistem secara otomatis memisahkan dan menghitung tepat 8 sel darah individual lengkap dengan Bounding Box hijau (Akurasi 100%).



Studi Kasus 7B (Biomedika): Penghitungan Sel Darah Menempel (Watershed)



Studi Kasus 7C (Agrikultur): Segmentasi Warna Buah Apel vs Jeruk (HSV)

Sortasi Buah Kamera Industri

• Masalah (Before): Pada ban berjalan (conveyor belt) pabrik pengemasan buah, kamera menangkap campuran buah apel merah dan jeruk oranye pada kondisi latar belakang pabrik.

• Solusi (Segmentasi Rentang Warna HSV):
  • Konversi RGB -> HSV untuk memisahkan komponen warna murni (Hue) dari intensitas cahaya (Value).
  • Masker Hue Merah Apel (0-12 & 165-180) vs Hue Oranye Jeruk (13-28).

• Hasil (After): Sistem secara simultan mendeteksi lokasi koordinat, menggambar Bounding Box, dan mengklasifikasikan jenis buah secara real-time.



Studi Kasus 7C (Agrikultur): Segmentasi Warna Buah Apel vs Jeruk (HSV)



Praktikum 7: Bedah Kode Python Thresholding & Watershed

Fungsi Segmentasi OpenCV

- 1. Otsu: cv2.threshold(..., cv2.THRESH_BINARY + cv2.THRESH_OTSU)
- 2. Adaptive: cv2.adaptiveThreshold(..., cv2.ADAPTIVE_THRESH_GAUSSIAN_C, ..., 19, 9)
- 3. Distance: cv2.distanceTransform(mask, cv2.DIST_L2, 5)
- 4. Watershed: cv2.watershed(img_bgr, markers)

Python: segmentation_watershed_demo.py

```
import cv2, numpy as np, matplotlib.pyplot as plt

img_bgr = cv2.imread('touching_cells.jpg')
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# 1. Otsu Thresholding Awal & Morfologi
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
clean = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# 2. Distance Transform untuk Mencari Inti Sel
dist = cv2.distanceTransform(clean, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)

# 3. Labeling Marker
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[clean == 0] = 0

# 4. Eksekusi Watershed
markers = cv2.watershed(img_bgr, markers)
img_bgr[markers == -1] = [0, 0, 255] # Garis batas merah pemisah

fig, axs = plt.subplots(1, 4, figsize=(15, 4))
axs[0].imshow(gray, cmap='gray'); axs[0].set_title('Original')
axs[1].imshow(clean, cmap='gray'); axs[1].set_title('Otsu Opening')
axs[2].imshow(dist, cmap='jet'); axs[2].set_title('Distance Transform')
axs[3].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)); axs[3].set_title('Watershed
Segmented')
for ax in axs: ax.axis('off')
plt.tight_layout(); plt.show()
```